

Azure Tenant-to-Tenant Secure Data Migration Notes (Terraform + Private Endpoint)

Overview

This project demonstrates a practical implementation of secure data migration between two Azure tenants using Infrastructure as Code (Terraform). The focus was on building a real-world enterprise-style architecture where security, network isolation, and controlled access are the key priorities.

Two separate Azure environments (Tenant A and Tenant B) were configured with Storage Accounts, and data was securely transferred between them using AzCopy along with SAS-based authentication and Private Endpoint connectivity.

Architecture Summary

The solution consists of:

- Two Azure tenants (Source and Destination)
- Azure Storage Accounts in both tenants
- Virtual Network (VNet) and Subnet configuration
- Private Endpoint for secure, private connectivity
- SAS (Shared Access Signature) for controlled access
- Client IP-based firewall restrictions
- AzCopy tool for data transfer

The entire infrastructure was deployed using Terraform to ensure consistency, repeatability, and automation.

Security Design Approach

Security was a major focus of this implementation. The following controls were applied:

Private Endpoint

A Private Endpoint was configured to ensure that traffic between the client and the storage account remains within the Azure backbone network. This eliminates exposure to the public internet and enhances data security.

SAS Token Access

SAS tokens were used to provide time-bound and permission-based access to storage resources. Permissions such as read, write, and list were explicitly defined to ensure least privilege access.

Network Firewall Rules

Client IP-based restrictions were applied at the storage account level to control access and prevent unauthorized connections.

Terraform Implementation

All infrastructure components were defined using Terraform, including:

- Resource groups
- Storage accounts
- Virtual networks and subnets
- Private endpoints
- Networking and access configurations

This approach ensured Infrastructure as Code (IaC) principles such as version control, automation, and environment consistency.

Data Migration Flow

1. Storage accounts created in both tenants
 2. Private network connectivity established using Private Endpoint
 3. SAS token generated with required permissions
 4. Client IP whitelisted in storage firewall rules
 5. Data transferred securely using AzCopy
 6. Destination storage account validated for successful upload
-

Key Learnings

- Understanding cross-tenant Azure architecture design
 - Practical implementation of Private Link and Private Endpoint
 - Deep dive into SAS token structure and permissions
 - Secure data transfer using AzCopy
 - Terraform-based infrastructure provisioning
 - Importance of DNS and network-level configurations in Private Link setups
-

Outcome

This project helped in understanding real-world cloud migration scenarios where security and controlled access are critical. It demonstrates how enterprise-level Azure architectures are designed using Infrastructure as Code and secure networking principles.

Conclusion

By combining Terraform automation, Private Endpoint security, and SAS-based access control, a secure and scalable data migration workflow was successfully implemented between two Azure tenants. This setup reflects real-world DevOps and Cloud Engineering practices used in production environments.